COMP4436 Artificial Intelligence of Things

Individual Project Specification

1. Overview

Real-world AIoT systems continuously receive data from sensors and must process the data in time to generate decisions before the information becomes stale. In this project, each student will design and evaluate a **streaming AIoT system** that processes sensor data and produces decisions under system constraints.

Achieving this target, the system must operate under several constraints:

- limited computing resources on edge devices
- network latency between edge and cloud
- continuous data arrival from sensors
- timeliness requirements for decisions

AIoT system designers aim to maximize overall system quality (**high accuracy** and **in-time inference**), while must consider trade-offs between:

- inference accuracy
- computation cost
- communication delay
- decision freshness

When the data arrival rate exceeds the system's processing capacity, the system must decide whether to reduce input data, drop queued samples, reduce computation cost, or offload computation to the cloud.

The goal of this project is to **explore AIoT system design and evaluation**, rather than simply building the most accurate AI model. Students will implement a prototype system and evaluate different design strategies.

This project accounts for **10% of the final course grade**.

2. System architecture

When data arrives faster than the system can process, the system must decide how much data to accept, how much computation to spend, and where the computation should run.

The system is composed of several functional components and data path, as shown in Figure 1. The functionality of each functional and data component is explained in Table 1.

Table 1: Explanation of system components

Component	Explanation	Implementation
Sensor Data	Data generated from sensors with a rate	MNIST
Sampling Rate Control	Simulating different sensor sampling rates (e.g., 5–50 samples per second)	Simulated
Admission Control	Data samples may be dropped if they become	Simulated

	stale or when the queue reaches its capacity	
Data Processing Queue	The queue of data samples to undergo inference. This queue can be combined with Data Input Queue	Data structure
Full AI model	Students may implement a small CNN model (e.g., LeNet-style) as the baseline model.	Actual Implementation
Model Compression	Offline component: generate compressed model variants by quantization or pruning	Actual Implementation
AI Inference	Executing the AI models over data samples from the Data Processing Queue. Two management techniques are used in case of insufficient compute capacity: dynamic model loading; distributed deployment	Actual Implementation + Simulation
System Parameter Simulation	To simulate management policy, system performance metrics (execution time, memory, etc.), network parameters	Mostly Simulated
System Parameters	Will be used in the decision of other functional components	Data structures
Quality Analysis	To collect and compute quality metrics, including but not limited to inference accuracy, data rate, deadline meet/miss ratio	Actual Implementation + Simulation

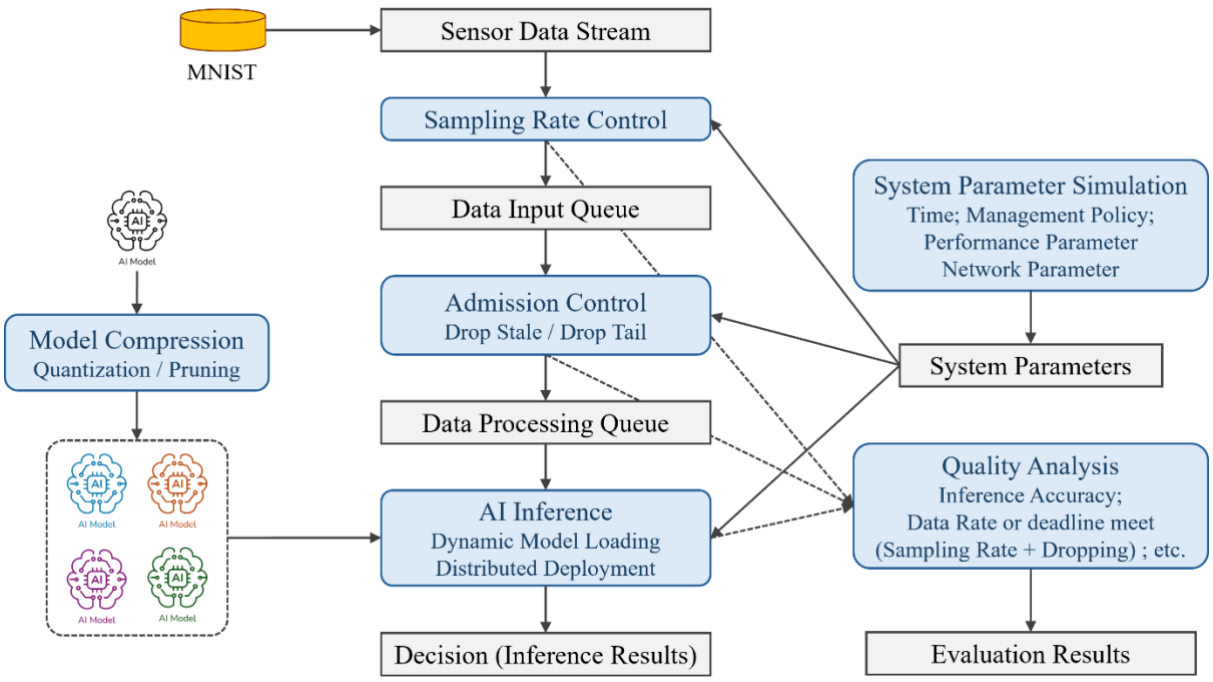

Figure 1: System architecture

3. Implementation requirements

The project should be implemented entirely in Python, preferably in a single notebook file.

Students may use frameworks such as PyTorch, TensorFlow, etc. Edge and cloud inference can be performed on the same computer, with simulated performance differences.

4. Required observations

Students must report on the following metrics:

- **End-to-end latency**: Time from data arrival to decision output. For each processed data sample, average latency and tail latency.
- **Data freshness**: Freshness measures how old the data is when the decision is produced, including individual and average metrics.
- **Queue behaviors**: Demonstrate how the processing queue evolves over time and how data dropping policies are applied.
- **Model metrics**: Accuracy and performance data (#MACs) of each model variant.
- **Accuracy-latency trade-off**: Logged information for individual samples and overall evaluation results. Model switching and distribution behaviors should also be demonstrated.

Students should demonstrate at least one scenario where the system becomes overloaded (arrival rate exceeds processing capacity). Students should not set system parameters to extreme values (thus the system only exhibits simple and fixed behavior).

5. Deliverables

1) **Project report**
   a) 4-5 pages excluding references. Be concise and do not run over the page limit.
   b) Suggested structure: Motivation, System design (overall architecture and components), Implementation, Experiments, and Discussion.
   c) Mandatory parts: system architecture diagram; experimental results; analysis of system trade-offs.
2) **Code submission**: Students must submit the source code used for experiments. The code should reproduce the results reported in the paper.
3) **Oral report session**: Each student will participate in a short explanation session, in around 5 minutes. Students should explain system architecture, scheduler strategy, experimental results, and observed system trade-offs. Students must understand their work, must be able to explain their implementation. Failure to explain the work during the explanation session may result in score reduction.

6. Academic integrity

This project must be completed individually. Students may discuss ideas with classmates, but code, report, experiments must be their own work. Academic misconduct will be handled according to university regulations.

7. Grading principles

Projects will be evaluated holistically based on the following aspects:

- **System design**: clarity of system architecture and scheduling strategy
- **Implementation**: correctness and completeness of the implemented system
- **Experimental evaluation**: quality of experiments and measurement of system metrics
- **Analysis and discussion**: understanding of system trade-offs
- **Oral explanation**: ability to clearly explain system design and experimental findings

Projects demonstrating strong understanding of AIoT system trade-offs will receive the highest evaluation. Projects will primarily be evaluated on the understanding of AIoT system behavior and trade-offs, rather than the complexity of the AI model.

8. Timeline

Deadline for all deliverables, including **project report, code, and presentation slides**, is: **23:59, 5 April 2026.**

Students are given sufficient time (approximately four weeks) to complete the project; therefore, **the deadline is firm (no late submission)**. Please do not risk the last hours of the deadline. Once the deliverables are submitted, no modification is allowed.

In-person presentations will be arranged in different time slots in **Week 13** (will use lecture sessions and lab sessions), since we have quite a lot of students (60 registered students). Further detailed time and location arrangements will be confirmed with all students.